

CloudGate

Target Architecture on Google Cloud

Step-by-step architecture documentation

A governed, AI-assisted application onboarding and migration-assessment platform. GenAI does the reading and drafting; deterministic work stays in code; a human approves at every gate.

12 August 2026

Prepared for leadership review · services subject to org GCP policy

1. Overview

CloudGate takes an application from readiness assessment through demand management, approval and design as a single governed workflow. It is a lightweight web front end over a workflow engine, with a document-understanding and generative-AI layer at its core, and API integrations to the systems of record. Every capability maps to an existing Google Cloud service, so the concept is built cloud-native with GenAI doing the reasoning and writing while deterministic work stays in code.

The diagram on the next page shows the full target architecture. This document then walks the end-to-end flow step by step, lists every service and its role, and states the design guardrails.

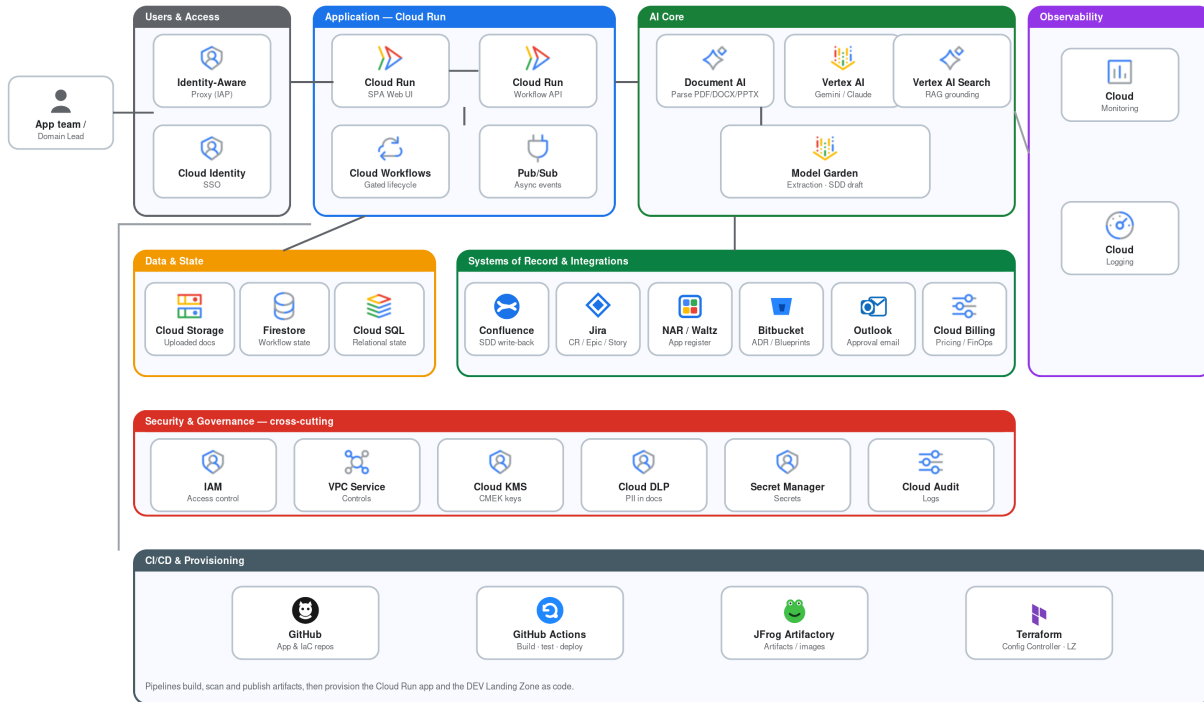
Architecture at a glance

Users & Access	Identity-Aware Proxy + Cloud Identity SSO in front of the app.
Application (Cloud Run)	Serverless UI and API, orchestrated by Cloud Workflows and Pub/Sub.
AI Core	Document AI parses uploads; Vertex AI (Gemini / Claude) extracts, maps, scores and drafts.
Grounding	Vertex AI Search (RAG) over approved ADRs & Blueprints in Bitbucket / Catalogue Page.
Data & State	Cloud Storage for documents; Firestore and Cloud SQL for workflow state.
Systems of Record	Confluence, Jira, NAR / Waltz, Outlook and Cloud Billing via API.
Security & Governance	IAM, VPC-SC, CMEK, DLP, Secret Manager and Audit Logs — cross-cutting.
Observability & Delivery	Cloud Monitoring / Logging; built and provisioned via GitHub, GitHub Actions, JFrog Artifactory and Terraform.

2. Target architecture diagram

CloudGate — Target Architecture on Google Cloud

Governed, AI-assisted application onboarding & migration-assessment platform — GenAI reasoning, deterministic work in code, human-in-the-loop approval gates.



An interactive version ([architecture.html](#)) offers a guided walkthrough that spotlights each layer in turn, plus hover detail on every tile. Icons are the official Google Cloud product and category icons.

3. End-to-end walkthrough

How a single application onboarding runs through the platform, from sign-in to delivery.

Step 1 — Secure entry

Services: [App team / Domain Lead](#) · [Identity-Aware Proxy \(IAP\)](#) · [Cloud Identity](#)

The app team member or Domain Lead opens the single-page web app. They never reach it directly — Identity-Aware Proxy enforces corporate SSO through Cloud Identity first, so access is controlled before anyone touches the platform.

Step 2 — Serverless app & gated workflow

Services: [Cloud Run \(Web UI + Workflow API\)](#) · [Cloud Workflows](#) · [Pub/Sub](#)

The UI and API run on Cloud Run — serverless, scaling to zero when idle. Cloud Workflows is the conductor that enforces the gated lifecycle (assessment → demand → approval → design). Pub/Sub carries events so slow work such as parsing or model calls runs asynchronously instead of blocking the user.

Step 3 — Upload & parse the document

Services: [Cloud Storage](#) · [Document AI](#)

The user uploads their architecture document; it lands in Cloud Storage. Document AI parses the PDF, DOCX or PPTX into structured text — headings, tables and content — that the model can reason over.

Step 4 — AI extraction, mapping & SDD draft

Services: [Vertex AI \(Gemini / Claude\)](#) · [Model Garden](#)

The core reasoning step. Vertex AI extracts the application's current-state profile, maps it to target GCP services, scores migration readiness, and drafts the SDD sections.

Step 5 — Grounded on approved references

Services: [Vertex AI Search \(RAG\)](#) · [Bitbucket](#) / [Catalogue Page](#)

Before writing anything, Vertex AI Search grounds the model on approved ADRs and Blueprints held in Bitbucket and the Catalogue Page. This is what stops the model inventing blueprint or ADR numbers — it can only cite real, approved decisions.

Step 6 — Application facts & cost

Services: [NAR / Waltz](#) · [Cloud Billing](#) / [Pricing](#)

Authoritative application data (the NAR register and application lookup) is pulled from NAR / Waltz, and provisional cost estimates come from the Cloud Billing pricing catalogue — provisional until real usage telemetry firms them up.

Step 7 — State & records

Services: [Firestore](#) · [Cloud SQL](#)

Every request, gap, approval and change-request state is written to Firestore, with Cloud SQL where relational storage is preferred — keeping the whole process reproducible.

Step 8 — Human gate & write-back

Services: Outlook · Confluence · Jira

The AI assists, it never decides. A human reviews and approves at each gate, prompted via Outlook (or in-app). Only then does the platform write the populated SDD back into Confluence and raise the Change Request, epics and stories in Jira.

Step 9 — Security & governance (cross-cutting)

Services: IAM · VPC-SC · Cloud KMS (CMEK) · Cloud DLP · Secret Manager · Audit Logs

This layer applies to every step above. IAM controls who can do what; VPC Service Controls draw a data perimeter; Cloud KMS/CMEK encrypts at rest with our own keys; Cloud DLP scans documents for PII; Secret Manager holds credentials; and Cloud Audit Logs record every action so the process is reproducible — important because architecture documents can contain sensitive data.

Step 10 — Observability & delivery

Services: Cloud Monitoring / Logging · GitHub · GitHub Actions · JFrog Artifactory · Terraform

Cloud Monitoring and Logging give dashboards, alerting and the operational trail. The platform itself ships as code: GitHub holds the app and IaC repos, GitHub Actions builds, tests and deploys the Cloud Run services, JFrog Artifactory stores build artifacts and images, and Terraform / Config Controller provisions the app and the DEV Landing Zone.

4. Service reference by layer

Every capability in the platform and the Google Cloud service or tool that delivers it.

Users & Access	Role
Identity-Aware Proxy (IAP)	Fronts the app; enforces authenticated, authorized access before Cloud Run.
Cloud Identity	Corporate single sign-on and user identity.
Application — Cloud Run	Role
Cloud Run (Web UI)	Serverless container hosting the single-page web app.
Cloud Run (Workflow API)	Serverless API for requests, gaps, approvals and change requests.
Cloud Workflows	Orchestrates the gated assessment → approval → design lifecycle.
Pub/Sub	Asynchronous eventing between workflow steps and services.
AI Core	Role
Document AI	Parses uploaded PDF / DOCX / PPTX into structured text.
Vertex AI (Gemini / Claude)	Extraction, current→target mapping, readiness scoring and SDD drafting.
Vertex AI Search (RAG)	Grounds recommendations on approved Blueprints, ERIs and ADRs.
Model Garden	Hosts the Gemini / Claude models used by Vertex AI.
Data & State	Role
Cloud Storage	Stores the uploaded architecture documents.
Firestore	Request, gap, approval and change-request workflow state.
Cloud SQL	Relational state where structured storage is preferred.

Systems of Record & Integrations	Role
Confluence API	Copies the Group Architecture template and writes back the populated SDD.
Jira API	Creates the Change Request, epics and stories.
NAR / Waltz	Authoritative application register and metadata (NAR register, app lookup).
Bitbucket / Catalogue Page	Source of approved ADRs and Blueprints used for RAG grounding.
Outlook (Exchange / Graph)	Heads-up and approval emails, or native in-app approvals.
Cloud Billing / Pricing	Pricing-driven FinOps cost and blueprint recommendation model.
Security & Governance (cross-cutting)	Role
Cloud IAM	Least-privilege access control across all services.
VPC Service Controls	Org perimeter to prevent data exfiltration.
Cloud KMS (CMEK)	Customer-managed encryption keys for data at rest.
Cloud DLP	Detects and protects PII inside uploaded documents.
Secret Manager	Stores API credentials and secrets.
Cloud Audit Logs	Full, reproducible audit trail of every action.
Observability	Role
Cloud Monitoring	Dashboards and alerting.
Cloud Logging	Operational logs and audit trail.
CI/CD & Provisioning	Role
GitHub	Source of truth for application and IaC repositories.
GitHub Actions	CI/CD pipelines that build, test and deploy the Cloud Run services.
JFrog Artifactory	Artifact and container-image registry for build outputs.
Terraform / Config Controller	Provisions the Cloud Run app and the DEV Landing Zone as code.

5. Design principles & guardrails

Human-in-the-loop gates	Readiness blockers and Domain Lead approval stay in the design so the model assists rather than decides.
Grounded, never fabricated	Every recommendation is grounded on the approved reference library (RAG); the model never invents ADR or blueprint numbers.
Costs are provisional	Cost estimates are treated as provisional until fed by real usage telemetry and the pricing API.
Sensitive data stays protected	Architecture documents can contain sensitive data, so Cloud DLP, VPC Service Controls, CMEK and full audit logging are enforced within the org boundary.
Auditable & reproducible	State lives in Firestore / Cloud SQL and every action in Cloud Audit Logs, so the whole process can be replayed and reviewed.

Suggested first step

Run a focused pilot on a single application: host the UI on Cloud Run behind IAP, wire Document AI plus Vertex AI for the extraction and SDD drafting, and measure the AI output against a human-produced baseline. This proves the core value quickly before investing in the full set of integrations.